



Algorithms & Data Structure

Sorting Algorithms

Dr. Ibraheem Al-Dhamari

ia@idhamari.com

5.5.2026

Note:

- This presentation and all resources are available publicly in my Github repository for free, please use the link below or scan the QR code
 - <https://github.com/ia-dev/algorithms>
- If you have questions in the future, feel free to use the discussion tab in the Github repository:
 - <https://github.com/ia-dev/algorithms/discussions>



Content

1. Terminology:

- Algorithms
- Sorting

2. Sorting algorithms:

- Bubble sort
- Selection sort
- Radix Sort

3. Resources

- Readings:
- Websites
- Video lectures

1. Terminology, Algorithms:

• What is algorithm?

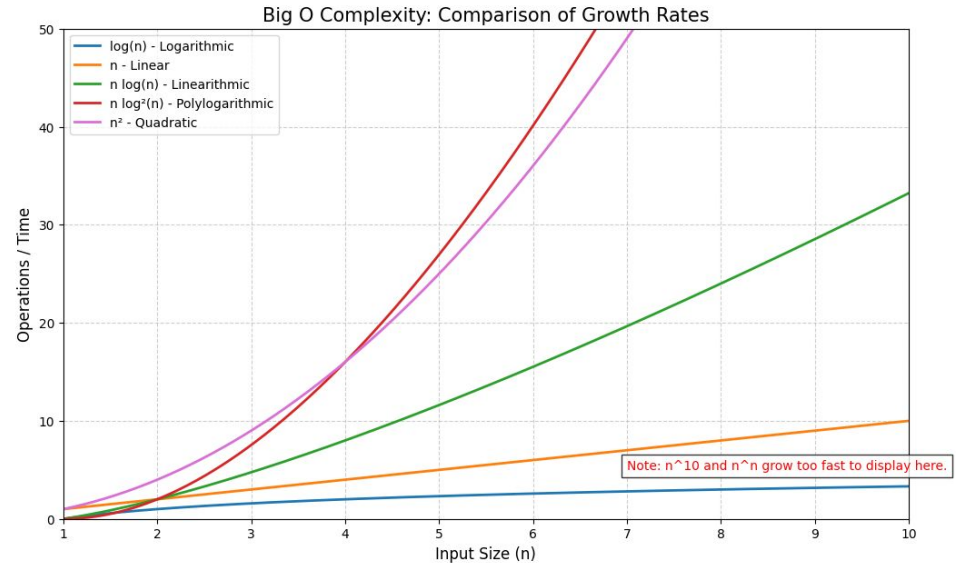
- A finite, deterministic, and effective problem-solving method suitable for implementation as a computer program.
- **Muhammad ibn Musa al-Khwarizmi** provided the first systematic, step-by-step instructions for simple math operations.
- **Pseudocode:** describing the algorithms steps in a way between natural language and a computer program.

• Why it is important?

- Practicaty e.g. Time and Memory.

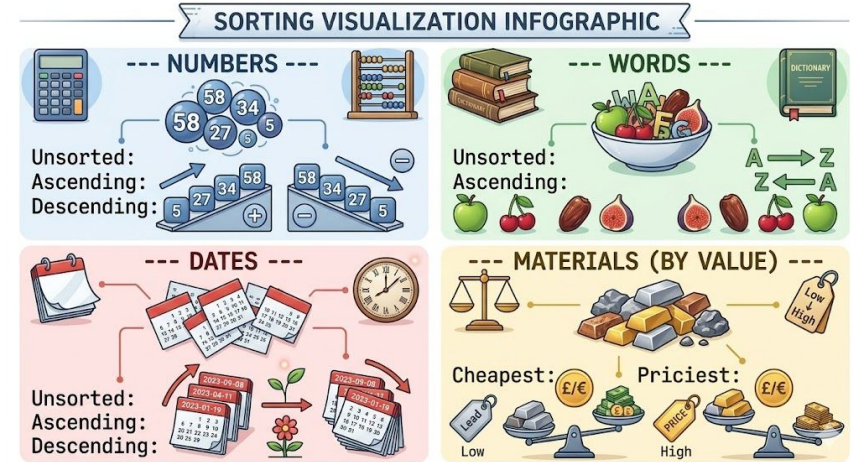
• How to compare algorithms?

- Using **Big O notation** (the worst case)
- **$O(\log(n))$** algorithm is better than **$O(n)$** algorithm
- **Big O** strip away constants and coefficients
- Focus on the "shape" of the scaling.



1. Terminology, Sorting:

- **What is sorting?**
 - Arranging a collection of data into a specific, meaningful logical order.
- **Why it is important?**
 - Making the data easier to search, analyze, or visualize.
- **Some popular sorting algorithms:**
 - Bubble sort
 - Selection sort
 - Insertion sort
 - Radix sort
 - Merge sort
 - Quick sort
 - Powersort



--- NUMBERS ---
Unsorted: [58, 34, 27, 5]
Ascending: [5, 27, 34, 58]
Descending: [58, 34, 27, 5]

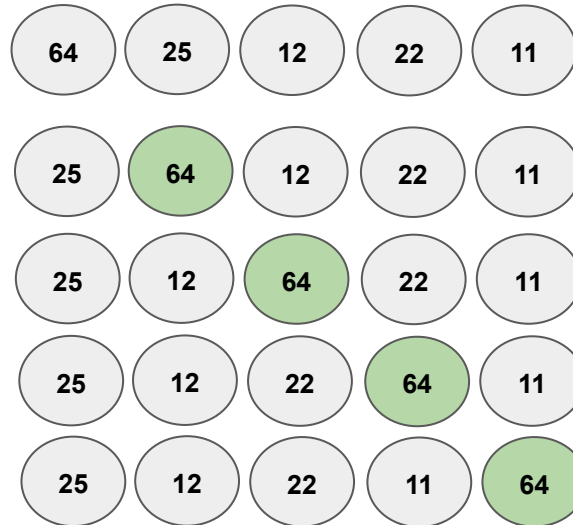
--- WORDS ---
Unsorted: ['Fig', 'Cherry', 'Apple', 'Date']
Ascending: ['Apple', 'Cherry', 'Date', 'Fig']
Descending: ['Fig', 'Date', 'Cherry', 'Apple']

--- DATES ---
Unsorted: ['2023-07-23', '2023-04-11', '2023-09-08', '2023-01-19']
Ascending: ['2023-01-19', '2023-04-11', '2023-07-23', '2023-09-08']
Descending: ['2023-09-08', '2023-07-23', '2023-04-11', '2023-01-19']

--- MATERIALS (BY VALUE) ---
Unsorted: ['Platinum', 'Silver', 'Gold', 'Copper', 'Lead']
Cheapest: ['Lead', 'Copper', 'Silver', 'Gold', 'Platinum']
Priciest: ['Platinum', 'Gold', 'Silver', 'Copper', 'Lead']

2. Sorting, Bubble Sort:

- A simple sorting algorithm that repeatedly compares adjacent elements, and swaps them if they are out of order.
- The largest element 'bubbles' to the top of the list.



Bubble Sort:

Algorithm : BubbleSort(L, n)

for $i \leftarrow 0$ to $n - 2$ do

for $j \leftarrow 0$ to $n - i - 2$ do

if $L[j] > L[j + 1]$ then

swap($L[j], L[j + 1]$)

$n = 5$

$i = 0$ to 3

$j = 0$ to $5 - i - 2$

$i = 0 \rightarrow 0$ to 3 four times

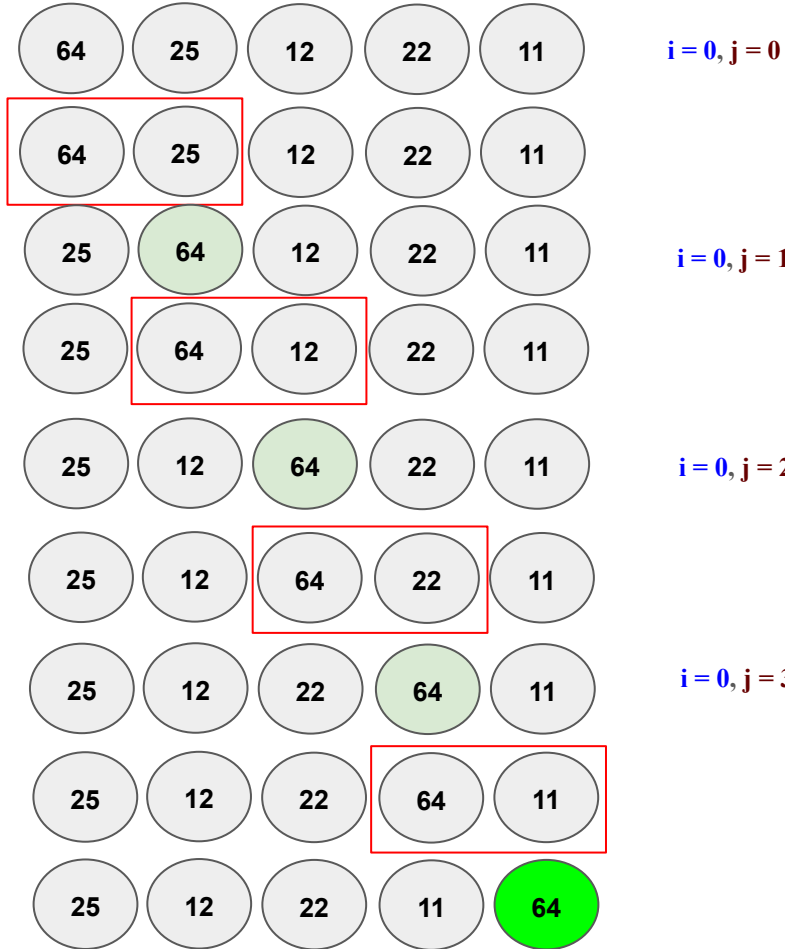
$i = 1 \rightarrow 0$ to 2 three times

$i = 2 \rightarrow 0$ to 1 two times

$i = 3 \rightarrow 0$ to 0 one time

Total = $4 + 3 + 2 + 1 = 10$ times

step	i	j	L[j]	L[j+1]
1	0	0	64	25
2	0	1	64	12
3	0	2	64	22
4	0	3	64	11



Bubble Sort:

Algorithm : BubbleSort(L, n)

for $i \leftarrow 0$ to $n - 2$ do

for $j \leftarrow 0$ to $n - i - 2$ do

if $L[j] > L[j + 1]$ then

swap($L[j], L[j + 1]$)

$n = 5$

$i = 0$ to 3

$j = 0$ to $5-i-2$

$i = 0 \rightarrow 0$ to 3 four times

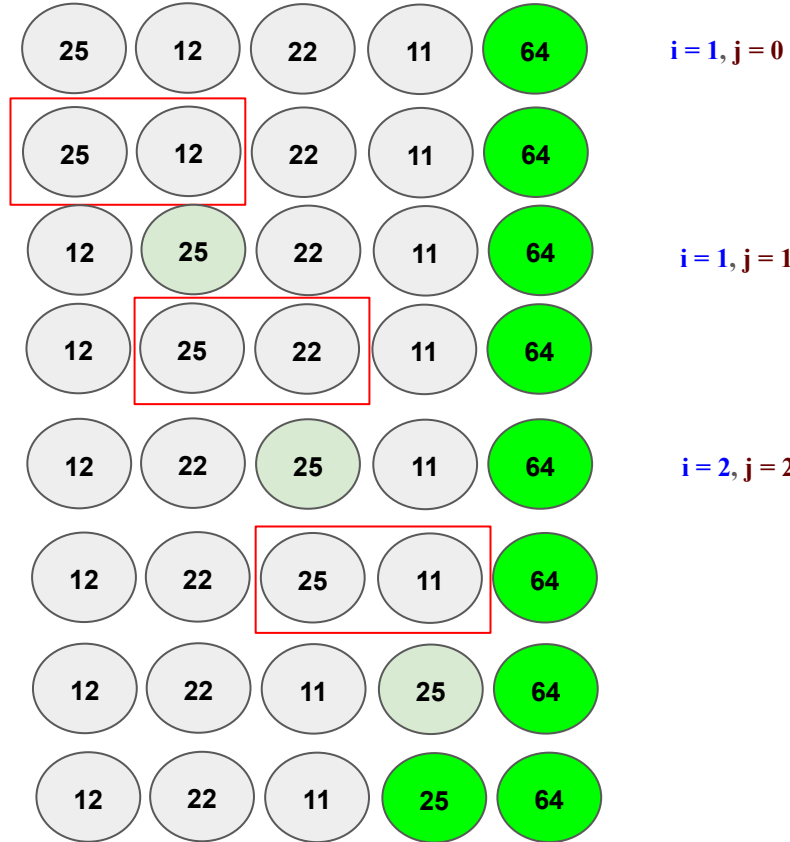
$i = 1 \rightarrow 0$ to 2 three times

$i = 2 \rightarrow 0$ to 1 two times

$i = 3 \rightarrow 0$ to 0 one time

Total = $4+3+2+1 = 10$ times

Idx	i	j	L[j]	L[j+1]
1	0	0	64	25
2	0	1	64	12
3	0	2	64	22
4	0	3	64	11
5	1	0	25	12
6	1	1	25	22
7	1	2	25	11



Bubble Sort:

Algorithm : BubbleSort(L, n)

for $i \leftarrow 0$ to $n - 2$ do

for $j \leftarrow 0$ to $n - i - 2$ do

if $L[j] > L[j + 1]$ then

swap($L[j], L[j + 1]$) /

$n = 5$

$i = 0$ to 3

$j = 0$ to 5-i-2

$i = 0 \rightarrow 0$ to 3 four times

$i = 1 \rightarrow 0$ to 2 three times

$i = 2 \rightarrow 0$ to 1 two times

$i = 3 \rightarrow 0$ to 0 one time

Total = 4+3+2+1 = 10 times

Idx	i	j	L[j]	L[j+1]
1	0	0	64	25
2	0	1	64	12
3	0	2	64	22
4	0	3	64	11
5	1	0	25	12
6	1	1	25	22
7	1	2	25	11
8	2	0	12	22
9	2	1	22	11



$i = 2, j = 0$

$i = 2, j = 1$

Bubble Sort:

Algorithm : BubbleSort(L, n)

for $i \leftarrow 0$ to $n - 2$ do

for $j \leftarrow 0$ to $n - i - 2$ do

if $L[j] > L[j + 1]$ then

swap($L[j], L[j + 1]$)

$n = 5$

$i = 0$ to 3

$j = 0$ to $5-i-2$

$i = 0 \rightarrow 0$ to 3 four times

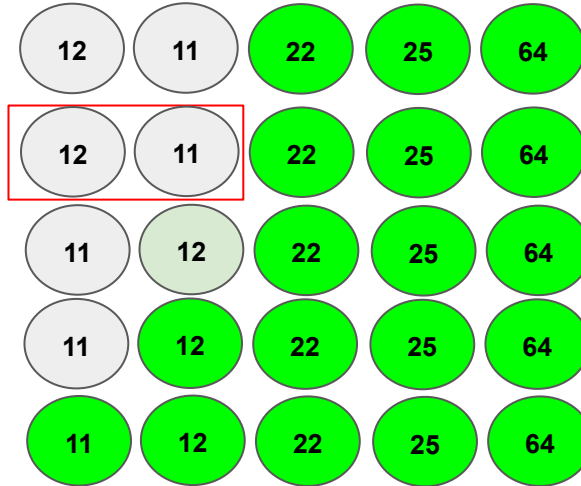
$i = 1 \rightarrow 0$ to 2 three times

$i = 2 \rightarrow 0$ to 1 two times

$i = 3 \rightarrow 0$ to 0 one time

Total = $4+3+2+1 = 10$ times

Idx	i	j	L[j]	L[j+1]
1	0	0	64	25
2	0	1	64	12
3	0	2	64	22
4	0	3	64	11
5	1	0	25	12
6	1	1	25	22
7	1	2	25	11
8	2	0	12	22
9	2	1	22	11
10	3	0	12	11



$i = 3, j = 0$



- **Time:**
 - $\Theta(n^2)$: Worst case.
 - $\Theta(n^2)$: Average case.
 - $\Theta(n^2)$: Best case.
- **Space:**
 - $O(1)$: In-place sorting.
- The number of comparisons is fixed $\geq$ the number of exchange.
- The data's initial order does not affect the algorithm performance.

Algorithm : BubbleSort(L, n)

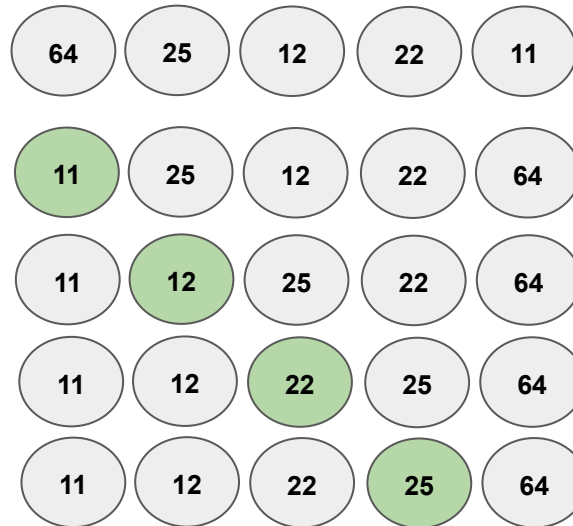
```

    for  $i \leftarrow 0$  to  $n - 2$  do
        for  $j \leftarrow 0$  to  $n - i - 2$  do
            if  $L[j] > L[j + 1]$  then
                swap( $L[j], L[j + 1]$ ) // Swap adjacent elements

```

2. Sorting, Selection Sort:

- One of the simplest sorting algorithms that works by repeatedly selecting the smallest remaining item.
- The smallest element is selected and placed in the beginning.



Selection Sort:

Algorithm : SelectionSort(L, n)

for $i \leftarrow 0$ to $n - 2$ **do:**

$min_idx \leftarrow i$

for $j \leftarrow i + 1$ to $n - 1$ **do:**

if $L[j] < L[min_idx]$ **then**

$min_idx \leftarrow j$

 swap($L[i], L[min_idx]$)

$n = 5$

$i = 0$ to 3

$j = i+1$ to 4

$i = 0 \rightarrow 1$ to 4 four times

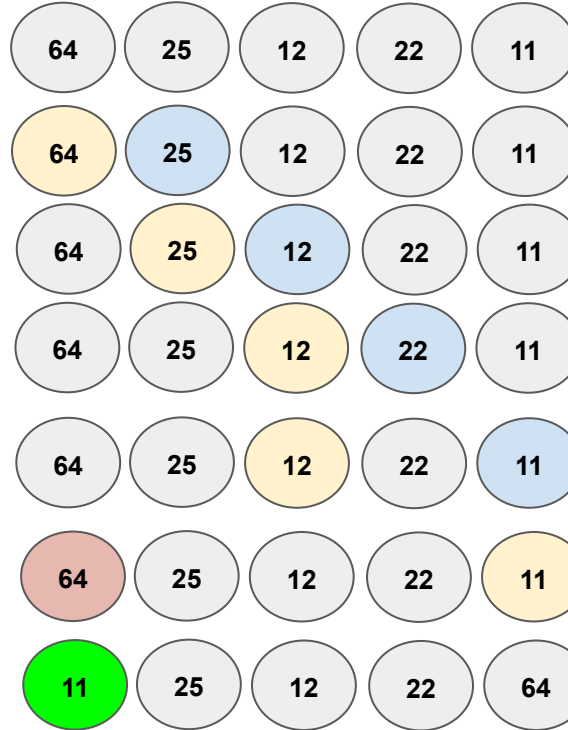
$i = 1 \rightarrow 2$ to 4 three times

$i = 2 \rightarrow 3$ to 4 two times

$i = 3 \rightarrow 4$ to 4 one time

Total = 4+3+2+1 = 10 times

step	i	j	m	L [j]	L [m]
1	0	1	0	25	64
2	0	2	1	12	25
3	0	3	2	22	12
4	0	4	2	11	12



$i = 0, j = 1, min_idx = 0 \rightarrow 1$

$i = 0, j = 2, min_idx = 1 \rightarrow 2$

$i = 0, j = 3, min_idx = 2$

$i = 0, j = 4, min_idx = 2 \rightarrow 4$

Selection Sort:

Algorithm : SelectionSort(L, n)

for $i \leftarrow 0$ to $n - 2$ **do:**

$min_idx \leftarrow i$

for $j \leftarrow i + 1$ to $n - 1$ **do:**

if $L[j] < L[min_idx]$ **then**

$min_idx \leftarrow j$

 swap($L[i], L[min_idx]$)

$n = 5$

$i = 0$ to 3

$j = i+1$ to 4

$i = 0 \rightarrow 1$ to 4 four times

$i = 1 \rightarrow 2$ to 4 three times

$i = 2 \rightarrow 3$ to 4 two times

$i = 3 \rightarrow 4$ to 4 one time

Total = $4+3+2+1 = 10$ times

step	i	j	m	L [j]	L [m]
1	0	1	0	25	64
2	0	2	1	12	25
3	0	3	2	22	12
4	0	4	2	11	12
5	1	2	1	12	25
6	1	3	2	22	12
7	1	4	2	64	12



$i = 1, j = 2, min_idx = 1 \rightarrow 2$

$i = 1, j = 3, min_idx = 2$

$i = 1, j = 4, min_idx = 2$

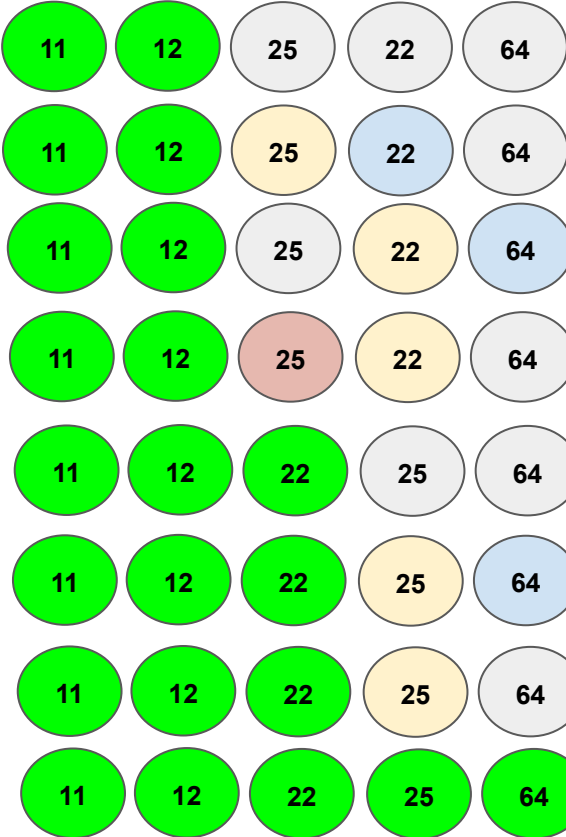
Selection Sort:

Algorithm : SelectionSort(L, n)

for $i \leftarrow 0$ to $n - 2$ do:
 $min_idx \leftarrow i$
 for $j \leftarrow i + 1$ to $n - 1$ do:
 if $L[j] < L[min_idx]$ then
 $min_idx \leftarrow j$
 swap($L[i], L[min_idx]$)

$n = 5$
 $i = 0$ to 3
 $j = i + 1$ to 4
 $i = 0 \rightarrow 1$ to 4 four times
 $i = 1 \rightarrow 2$ to 4 three times
 $i = 2 \rightarrow 3$ to 4 two times
 $i = 3 \rightarrow 4$ to 4 one time
Total = $4 + 3 + 2 + 1 = 10$ times

step	i	j	m	L [j]	L [m]
1	0	1	0	25	64
2	0	2	1	12	25
3	0	3	2	22	12
4	0	4	2	11	12
5	1	2	1	12	25
6	1	3	2	22	12
7	1	4	2	64	12
8	2	3	2	22	25
9	2	4	3	64	22
10	3	4	3	64	25



$i = 2, j = 3, min_idx = 2 \rightarrow 3$

$i = 2, j = 4, min_idx = 3$

$i = 3, j = 4, min_idx = 3$



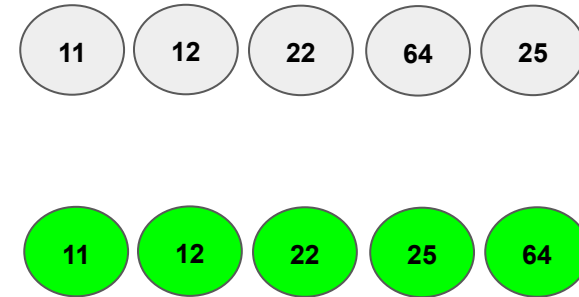
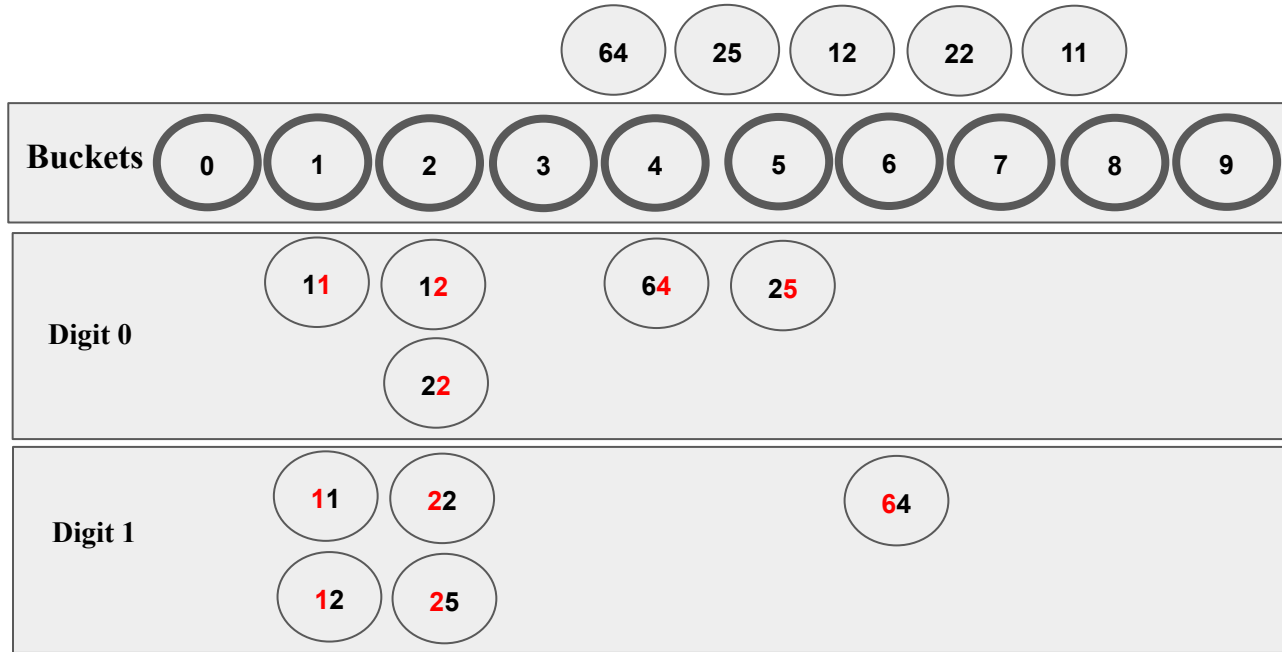
- **Time:**
 - $\Theta(n^2)$: Worst case.
 - $\Theta(n^2)$: Average case.
 - $\Theta(n^2)$: Best case.
- **Space:**
 - $O(1)$: In-place sorting.
- The number of comparisons is fixed
- The number of exchange is much smaller than the bubble sort.
 - $O(n)$ vs $O(n^2)$
- The data's initial order does not affect the algorithm performance.

Algorithm : SelectionSort(L, n)

for $i \leftarrow 0$ to $n - 2$ **do**: $\text{min_idx} \leftarrow i$ **for** $j \leftarrow i + 1$ to $n - 1$ **do**:**if** $L[j] < L[\text{min_idx}]$ **then** $\text{min_idx} \leftarrow j$ $\text{swap}(L[i], L[\text{min_idx}])$

2. Sorting, Radix Sort:

- Non-comparative integer sorting algorithm that sorts data with integer keys by grouping the keys by individual digits that share the same significant position and value.
- Highly efficient but requires known small maximum number of digits.



Algorithm : RadixSort(L, n)

 $\text{max_val} \leftarrow \max(L)$
 $\text{exp} \leftarrow 1$
while $\text{max_val} \div \text{exp} > 0$ **do**

 CountingSort(L, n, exp)

 $\text{exp} \leftarrow \text{exp} \times 10$
return L

Pass	exp Value	Math (Number // exp)	Modulo (% 10)	Isolated Digit
1st Pass	exp = 1	256 // 1 = 256	256 % 10	6 (Ones place)
2nd Pass	exp = 10	256 // 10 = 25	25 % 10	5 (Tens place)
3rd Pass	exp = 100	256 // 100 = 2	2 % 10	2 (Hundreds place)

Algorithm : CountingSort(L, n, exp)

 output $\leftarrow$ array of size n filled with 0

 count $\leftarrow$ array of size 10 filled with 0

for $i \leftarrow 0$ to $n - 1$ **do**

 index $\leftarrow L[i] \div \text{exp}$

 count[index mod 10] $\leftarrow$ count[index mod 10] + 1

for $i \leftarrow 1$ to 9 **do**

 count[i] $\leftarrow$ count[i] + count[i - 1]

 $i \leftarrow n - 1$
while $i \geq 0$ **do**

 index $\leftarrow L[i] \div \text{exp}$

 output[count[index mod 10] - 1] $\leftarrow L[i]$

 count[index mod 10] $\leftarrow$ count[index mod 10] - 1

 $i \leftarrow i - 1$
for $i \leftarrow 0$ to $n - 1$ **do**

 $L[i] \leftarrow \text{output}[i]$

```

Digit place 1
Initial values
    L    [64, 25, 12, 22, 11]
    output [0, 0, 0, 0, 0]
    count [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Store count of occurrences
i: 0 index 64
count [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
i: 1 index 25
count [0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0]
i: 2 index 12
count [0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0]
i: 3 index 22
count [0, 0, 2, 0, 0, 1, 1, 0, 0, 0, 0]
i: 4 index 11
count [0, 1, 2, 0, 0, 1, 1, 0, 0, 0, 0]
Change count[i] so that it contains the actual position
i: 1 count: [0, 1, 2, 0, 0, 1, 1, 0, 0, 0, 0]
i: 2 count: [0, 1, 3, 0, 0, 1, 1, 0, 0, 0, 0]
i: 3 count: [0, 1, 3, 3, 0, 1, 1, 0, 0, 0, 0]
i: 4 count: [0, 1, 3, 3, 4, 1, 0, 0, 0, 0, 0]
i: 5 count: [0, 1, 3, 3, 4, 5, 0, 0, 0, 0, 0]
i: 6 count: [0, 1, 3, 3, 4, 5, 5, 0, 0, 0, 0]
i: 7 count: [0, 1, 3, 3, 4, 5, 5, 5, 0, 0, 0]
i: 8 count: [0, 1, 3, 3, 4, 5, 5, 5, 5, 0, 0]
i: 9 count: [0, 1, 3, 3, 4, 5, 5, 5, 5, 5, 5]
Build the output array
i: 3 index 11
output [11, 0, 0, 0, 0]
count [0, 0, 3, 3, 4, 5, 5, 5, 5, 5, 5]
i: 2 index 22
output [11, 0, 22, 0, 0]
count [0, 0, 2, 3, 4, 5, 5, 5, 5, 5, 5]
i: 1 index 12
output [11, 12, 22, 0, 0]
count [0, 0, 1, 3, 4, 5, 5, 5, 5, 5, 5]
i: 0 index 25
output [11, 12, 22, 0, 25]
count [0, 0, 1, 3, 4, 4, 5, 5, 5, 5, 5]
i: -1 index 64
output [11, 12, 22, 64, 25]
count [0, 0, 1, 3, 3, 4, 5, 5, 5, 5, 5]
List after sorting by digit place 1: [11, 12, 22, 64, 25]

```

```

Digit place 10
Initial values
    L    [11, 12, 22, 64, 25]
    output [0, 0, 0, 0, 0]
    count [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Store count of occurrences
i: 0 index 1
count [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
i: 1 index 1
count [0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0]
i: 2 index 2
count [0, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0]
i: 3 index 6
count [0, 2, 1, 0, 0, 0, 0, 1, 0, 0, 0]
i: 4 index 2
count [0, 2, 2, 0, 0, 0, 0, 1, 0, 0, 0]
Change count[i] so that it contains the actual position
i: 1 count: [0, 2, 2, 0, 0, 0, 0, 1, 0, 0, 0]
i: 2 count: [0, 2, 4, 0, 0, 0, 0, 1, 0, 0, 0]
i: 3 count: [0, 2, 4, 4, 0, 0, 0, 1, 0, 0, 0]
i: 4 count: [0, 2, 4, 4, 4, 0, 0, 1, 0, 0, 0]
i: 5 count: [0, 2, 4, 4, 4, 4, 1, 0, 0, 0, 0]
i: 6 count: [0, 2, 4, 4, 4, 4, 4, 5, 0, 0, 0]
i: 7 count: [0, 2, 4, 4, 4, 4, 4, 5, 5, 0, 0]
i: 8 count: [0, 2, 4, 4, 4, 4, 4, 5, 5, 5, 0]
i: 9 count: [0, 2, 4, 4, 4, 4, 4, 5, 5, 5, 5]
Build the output array
i: 3 index 2
output [0, 0, 0, 25, 0]
count [0, 2, 3, 4, 4, 4, 5, 5, 5, 5, 5]
i: 2 index 6
output [0, 0, 0, 25, 64]
count [0, 2, 3, 4, 4, 4, 4, 5, 5, 5, 5]
i: 1 index 2
output [0, 0, 22, 25, 64]
count [0, 2, 2, 4, 4, 4, 4, 4, 5, 5, 5]
i: 0 index 1
output [0, 12, 22, 25, 64]
count [0, 1, 2, 4, 4, 4, 4, 4, 5, 5, 5]
i: -1 index 1
output [11, 12, 22, 25, 64]
count [0, 0, 2, 4, 4, 4, 4, 4, 5, 5, 5]
List after sorting by digit place 10: [11, 12, 22, 25, 64]

```

- **Time:**
 - $\Theta(d(n+k))$: Worst case.
 - $\Theta(d(n+k))$: Average case.
 - $\Theta(d(n+k))$: Best case.
- **Space:**
 - $O(n+k)$: Not in-place sorting.
- The data's initial order does not affect the algorithm performance.

Algorithm : RadixSort(L, n)

```

max_val ← max(L)
exp ← 1

while max_val ÷ exp > 0 do
    CountingSort( $L, n, exp$ )
    exp ← exp × 10

return L

```

Algorithm : CountingSort(L, n, exp)

```

output ← array of size  $n$  filled with 0
count ← array of size 10 filled with 0

for  $i \leftarrow 0$  to  $n - 1$  do
    index ←  $L[i] \div exp$ 
    count[index mod 10] ← count[index mod 10] + 1

for  $i \leftarrow 1$  to 9 do
    count[i] ← count[i] + count[i - 1]

 $i \leftarrow n - 1$ 
while  $i \geq 0$  do
    index ←  $L[i] \div exp$ 
    output[count[index mod 10] - 1] ←  $L[i]$ 
    count[index mod 10] ← count[index mod 10] - 1
     $i \leftarrow i - 1$ 

for  $i \leftarrow 0$  to  $n - 1$  do
     $L[i] \leftarrow output[i]$ 

```

2. Sorting, SOTA:

- There are more efficient sorting algorithms with $O(n \log(n))$ e.g.
 - Merge Sort.
 - Quick Sort.
 - Power Sort.

Please see the shared Colab Notebook for more information!

```
--- Sorting Algorithm Performance Comparison (n=1000) ---
Algorithm          | Time (seconds)
-----
Quick Sort         | 0.001245
Radix Sort         | 0.001903
Shell Sort         | 0.001972
Merge Sort         | 0.002133
Power Sort         | 0.033964
Insertion Sort     | 0.038142
Selection Sort     | 0.041405
Bubble Sort        | 0.061756
```

3. Resources:

- Popular books

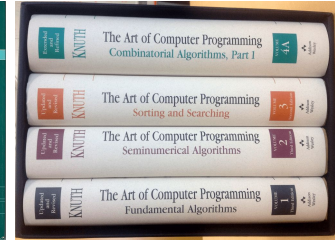
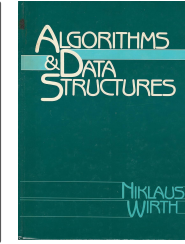
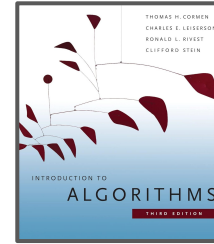
- Introduction to Algorithms, Cormen
- Algorithms & Data Structure, Wirth
- The Art of Computer Programming, Knuth

- Animations

- [Advanced Sorting Algorithm Visualizer](#)
- [Visualgo Sorting Animation](#)

- Online courses e.g. MIT:

- [Introduction to computational thinking](#)
- [Introduction to algorithms](#)
- [Design and analysis of algorithms](#)
- [Introduction to the Theory of Computation](#)



*Thank
You*

